MASTER REVISION GUIDE

JavaScript & APIs
Concept Recall Handbook

Aapke sabhi covered concepts ka crystal-clear summary, visual examples, aur Roman Urdu explanations — dimagh mein concepts pakkay karne ke liye.

Table of Contents / Index:

01 JavaScript Fundamentals

02 Asynchronous JavaScript

03 Fetch API & HTTP Methods

04 REST API Architecture

05 ES6+ Modern Essentials

06 Quick Recall Cheat Sheet

1. JavaScript Fundamentals

CORE CONCEPTS

1.1 Variables: var vs let vs const

DIMAAG ME YAAD RAKHNE KI TRICK:
`var` purana hai (function scoped, re-declare ho jata hai, bugs paida karta hai). Hamesha `const` use karein default ke tor par; agar value baad me badalni ho tabhi `let` lagayein. Dono block-scoped hote hain `{ }`.

Keyword	Scope	Re-assign	Re-declare
`var`	Function Scope	Yes	Yes (Dangerous)
`let`	Block Scope { }	Yes	No
`const`	Block Scope { }	No	No

1.2 Scope & Closures

CLOSURE KYA HOTA HAI?
Jab ek inner function apne parent (outer) function ke variables ko us waqt bhi yaad rakhta hai jab parent function execute hokar finish ho chuka ho. Isko **Closure** kehte hain.

```
function createCounter() {
    let count = 0;
    return function() {
        count++;
        return count;
    };
}
const counter = createCounter();
counter(); // Output: 1
counter(); // Output: 2 (count retain raha!)
```

1.3 Essential Array Methods (map, filter, reduce)

FARQ AUR ISTEMAAL:

- **map():** Har item ko badal kar same size ka new array banata hai.
- **filter():** Condition match karne wale items ka chhota/barabar array banata hai.
- **reduce():** Saare items ko combine karke single value (total/object) nikalta hai.

```
const nums = [10, 20, 30];
const doubled = nums.map(n ⇒ n * 2);         // [20, 40, 60]
const above15 = nums.filter(n ⇒ n > 15);      // [20, 30]
const total = nums.reduce((acc, curr) ⇒ acc + curr, 0); // 60
```

2. Asynchronous JavaScript

PROMISES & ASYNC

2.1 Callbacks aur Callback Hell

CONCEPT:
Callback ek function hota hai jo doosre function ko as an argument pass hota hai taake kaam khatam hone par run ho. Jab ek ke andar ek bohot se callbacks aane lagein to usay "Pyramid of Doom" ya Callback Hell bolte hain. Isko khatam karne ke liye Promises aaye!

2.2 Promises: States aur Methods

PROMISE KI 3 STATES:
1. **Pending:** Kaam chal raha hai. 2. **Fulfilled:** Kaam kamiyab (`resolve()` call hua → `.then()` chalega). 3. **Rejected:** Error agaya (`reject()` call hua → `.catch()` chalega).

```
const checkStock = new Promise((resolve, reject) => {
    let inStock = true;
    if (inStock) resolve("Order Delivered");
    else reject("Out of Stock");
});

checkStock
    .then(msg => console.log(msg))
    .catch(err => console.log(err))
    .finally(() => console.log("Har haal me chalega"));
```

2.3 Promise Combinators Comparison

Combinator	Kab resolve hota hai?	Kab reject hota hai?
Promise.all	Jab SARE promises resolve hon	Agar EK BHI reject ho jaye (Fast fail)
Promise.allSettled	SARE complete hon (resolve/reject dono report milte hain)	Kabhi reject nahi hota
Promise.race	Jo pehla settle ho (chahe resolve ho ya reject)	Jo pehla settle ho agar reject nikle
Promise.any	Jo PEHLA resolve ho jaye	Sirf tab agar SARE ke SARE reject hon

2.4 Async / Await with Try / Catch

> **CLEANEST WAY:**
> Yeh promises ko synchronous code ki tarah likhne ka syntax sugar hai. `await` sirf `async` function ke andar kaam karta hai aur promise resolve hone ka intezar karta hai.

```
async function loadData() {
    try {
        let response = await fetch("https://api.example.com/data");
        if (!response.ok) throw new Error("Status " + response.status);
        let data = await response.json();
        console.log(data);
    } catch (error) {
        console.log("Catched Error:", error.message);
    }
}
```

3. Fetch API & HTTP Requests

NETWORK & APIS

3.1 HTTP Methods (CRUD Operations)

Method	CRUD Action	Description / Use Case
GET	Read	Server se data mangwana (koi body nahi hoti)
POST	Create	Server par naya record bhejna (body me JSON data hota hai)
PUT	Update (Full)	Poore object ko nayi values se replace karna
PATCH	Update (Partial)	Sirf specific field update karna (e.g. sirf email)
DELETE	Delete	Server se record remove karna

3.2 HTTP Status Codes Quick Sheet

INTERVIEW TIPS:

- **200 OK:** Request kamiyab hui.
- **201 Created:** Naya resource ban gaya (POST me milta hai).
- **400 Bad Request:** Client ne data ghalat bheja.
- **401 Unauthorized:** Login/Token missing hai.
- **404 Not Found:** URL ya resource mojood nahi.
- **500 Internal Server Error:** Server ke code me masla hai.

3.3 Headers & JSON Handling

> `JSON.stringify(obj)` JavaScript object ko text me badalta hai taake server ko bheja ja sake. `response.json()` server ke text ko wapis JS object me convert karta hai.

```
const res = await fetch("https://jsonplaceholder.typicode.com/posts", {
    method: "POST",
    headers: {
        "Content-Type": "application/json", // Server ko batata hai hum JSON bhej rahe
        "Accept": "application/json"
    },
    body: JSON.stringify({ title: "Title", body: "Content", userId: 1 })
});
```

4. REST API Concepts

ARCHITECTURE

Route Parameters vs Query Parameters

Type	Example URL	Kab use hota hai?
Route Parameter (Path Variable)	`/users/5` `/posts/102`	Kisi **specific ID** ya single unique resource ko identify karne ke liye.
Query Parameter (Query String)	`/users?_limit=5` `/products?category=shoes&page=2`	Data ko **filter, search, sort, ya paginate** karne ke liye (`?key=value&key2=val`).

5. ES6+ Modern JavaScript

MODERN SYNTAX

5.1 Destructuring & Spread Operator

```
// Destructuring (Directly extracting values)
const user = { name: "Ahmed", age: 22, city: "Lahore" };
const { name, age } = user;

// Spread Operator ... (Copying or merging)
const nums = [1, 2];
const moreNums = [ ...nums, 3, 4]; // [1, 2, 3, 4]
const updatedUser = {  ...user, age: 23 };
```

5.2 Optional Chaining (?.) & Nullish Coalescing (??)

CRASH SE BACHNE KE TOOLS:

- `?.` Agar property mojood na ho to error throw karke app crash nahi karta, balkay `undefined` return karta hai.
- `??` Sirf tab fallback deta hai jab value `null` ya `undefined` ho (0 ya false ko valid manta hai).

```
const data = { user: { profile: null } };
console.log(data.user.profile?.avatar); // undefined (No error!)

let userScore = 0;
let score = userScore ?? 10; // Output: 0 ( || hota toh 10 kardeta!)
```

5.3 ES Modules: import & export

```
// math.js
export function add(a, b) { return a + b; }
export const PI = 3.14;

// app.js
import { add, PI } from "./math.js";
console.log(add(5, 10));
```

> **Note:** Node.js me ES modules use karne ke liye `package.json` me `"type": "module"` likhna zaroori hota hai.

Daily Revision Strategy (Aapke liye)

PRACTICE FORMULA:

1. Har roz 1 folder pakrein (e.g. `02-asynchronous`).
2. File ka code bina dekhe apne lafzon me run karein.
3. Error aaye to console error read karein aur fix karein.
4. Jab code chal jaye to JSONPlaceholder ke sath real test karein!